

Programação Estruturada em C++

Prof. Ms. *Rogério Ad. Sousa*

Índice

- Vetores e Matrizes
 - Definição
- Vetores
 - Declaração e Inicialização
 - Exemplos
- Matrizes
 - Declaração e Inicialização
 - Exemplos
- Atividades

Definição

- Vetores e matrizes são variáveis compostas homogêneas, ou seja, são agrupamentos de dados que individualmente ocupam o mesmo tamanho na memória e são referenciados pelo mesmo nome, geralmente são individualizadas usando-se índices.
- Vetores distinguem-se das matrizes apenas pela característica de ter dimensão $(1 \times n)$ ou $(n \times 1)$, essencialmente vetores são matrizes linha ou matrizes coluna.
- Em linguagem "C++" vetores e matrizes são usados abundantemente para compor estruturas de dados necessárias para composição de diversos recursos.
- Esta usa, mais explicitamente, vetores de caracteres para definir cadeias de texto, o que é conhecido como o mais trivial uso de vetores. Além deste recurso, o "C++" também define meio de criação de matrizes tipo $(n \times m)$, provendo, desta forma os recursos necessários para criação destes conjuntos de dados.

Vetores

- Considere o seguinte programa. Este programa pede ao Usuário notas de 4 estudantes, calcula a média e imprime as notas e a média.

```
int main()
{
    int nota0, nota1, nota2, nota3;
    int media;

    cout << "Entre a nota do estudante 0: ";
    cin >> nota0;
    cout << "Entre a nota do estudante 1: ";
    cin >> nota1;
    cout << "Entre a nota do estudante 2: ";
    cin >> nota2;
    cout << "Entre a nota do estudante 3: ";
    cin >> nota3;

    media = (nota0 + nota1 + nota2 + nota3) / 4;

    cout << "Notas: " << nota0 << " " << nota1 << " " << nota2 << " "
        << nota3 << endl;
    cout << "Media: " << media << endl;
}
```

Vetores

- O programa é bem simples, mas ele tem um problema.
- O que acontece se o número de estudantes aumentar ?
 - O programa ficaria muito grande.
 - Imagine o mesmo programa se existissem 100 estudantes.
- O que precisamos é uma abstração de dados para agrupar dados relacionados. Este é o objetivo de *arrays* em C++ .
- Um *array* é uma coleção de um ou mais objetos, do mesmo tipo, armazenados em endereços adjacentes de memória.
- Cada objeto é chamado de *elemento* do array. Da mesma forma que para variáveis simples, damos um nome ao array.
- O tamanho do array é o seu número de elementos. Cada elemento do array é numerado, usando um inteiro chamado de *índice*.
- Em C++ , a numeração começa com 0 e aumenta de um em um. Assim, o último índice é igual ao número de elementos do array menos um.
- Por exemplo, podemos definir um array nota de tamanho 100 para armazenar as notas dos cem estudantes:
int nota[100];
- Quando o compilador encontra esta definição, ele aloca 200 bytes consecutivos de memória (dois bytes – referente a cada int – para cada nota). Cada nota pode ser acessada dando o nome do array e o índice entre colchetes:
 - como nota[0] (para a primeira nota), nota[1] para a segunda nota, e assim por diante, até a última nota, nota[99].

Declaração e inicialização de Vetores

- Os arrays podem ser inicializados quando são definidos. Se o array não for inicializado, então ele contém valores indefinidos (também conhecidos como *lixo*).
- Para inicializar um array, um valor para cada elemento deve ser especificado. Estes valores devem estar entre chaves ({ e }) e são separados por vírgula (.). Alguns exemplos:

```
int valor[4] = { 1, 42, -13, 273 };
```

```
// o tamanho do array pode ser omitido  
int peso[] = { 153, 135, 170 };
```

- No primeiro exemplo, valor é um array de 4 inteiros onde valor[0] é 1, valor[1] é 42, valor[2] é -13, e valor[3] é 273.
- Note que no segundo exemplo, o tamanho do array foi omitido. Neste caso, o compilador calcula o tamanho como sendo o número de elementos listados. Quando um array é definido, se ele não for inicializado, o tamanho do array deve ser especificado. Se o array for inicializado, o tamanho pode ser omitido. O segundo exemplo acima é equivalente a : `int peso[3] = { 153, 135, 170 };`

Exemplo

- Com o uso de vetor o exemplo anterior ficaria desta forma:

```
int main()
{
    int indice, nota[4];
    float total;

    for(indice = 0; indice < 4; indice += 1)
    {
        cout << "Entre a nota do estudante " << indice << ": ";
        cin >> nota[indice];
    }

    cout << "Notas: ";

    total = 0;
    for(indice = 0; indice < 4; indice += 1)
    {
        cout << nota[indice] << " ";
        total = total + nota[indice];
    }
    cout << endl << "Media: " << total / 4 << endl;
}
```

Declaração e inicialização de Matrizes

- Anteriormente apresentamos arrays unidimensionais. Em C++ , é possível também definir arrays com 2 ou mais dimensões. Eles são arrays de arrays. Um array de duas dimensões podem ser imaginado como uma matriz (ou uma tabela).
- Para definir e acessar arrays de dimensões maiores, usamos colchetes adicionais ([e]).
- Por exemplo:

```
int tabela[3][5];
```

- Define um array bidimensional chamado tabela que é uma matriz 3 por 5 de valores do tipo int (15 valores no total). Os índices da primeira dimensão vão de 0 a 2, e os índices da segunda dimensão vão de 0 a 4.

Declaração e inicialização de Matrizes

- Arrays multidimensionais podem ser inicializados usando listas aninhadas de elementos entre chaves. Por exemplo, um array bidimensional tabela com três linhas e duas colunas pode ser inicializado da seguinte forma:

```
double tabela[3][2] = { {1.0,  0.0},    // linha 0
                        {-7.8, 1.3},    // linha 1
                        {6.5,  0.0}      // linha 2
                      };
```

- Quando o array é inicializado, o tamanho da primeira dimensão pode ser omitido. A definição de array abaixo é equivalente a dada anteriormente.

```
double tabela[][2] = { {1.0,  0.0},    // linha 0
                       {-7.8, 1.3},    // linha 1
                       {6.5,  0.0}      // linha 2
                     };
```

Exemplo Matrizes

- Programa que imprime os elementos de um array bidimensional.

```
#include <iostream>
using namespace std;

#define ALTURA 5
#define LARGURA 5

int main()
{
    int x;                // numero da coluna
    int y;                // numero da linha
    char matriz [ALTURA] [LARGURA]; // array 2-D [num_lins, num_co]

    // preenche a matriz com zeros

    for(y = 0; y < ALTURA; y += 1)
    {
        for(x = 0; x < LARGURA; x += 1)
        {
            matriz[y][x] = 0;
        }
    }

    // Imprime a matriz com zeros e a coordenada escolhida com 1

    cout << endl << "Entre coordenadas na forma \"y x\"." << endl;
    cout << "Use valores negativos para sair do programa." << endl;

    cout << "Coordenadas: ";
    cin >> y >> x;

    while (x >= 0 && y >= 0)
    {
        matriz[y][x] = 1; // coloca 1 no elemento escolhido

        for(y = 0; y < ALTURA; y += 1) // imprime o array todo
```

```
    {
        for(x = 0; x < LARGURA; x += 1)
        {
            cout << matriz[y][x] << " ";
        }
        cout << endl << endl;
    }

    cout << endl;
    cout << "Coordenadas: ";
    cin >> y >> x;
}
```

Exemplo Matrizes

- O exemplo abaixo preenche os elementos de um array bidimensional com os valores que representam a tabuada e imprime a matriz.

```
#include <iostream>
#include <iomanip>
using namespace std;

#define LIN 10
#define COL 10

int main()
{
    int x;                // numero da coluna
    int y;                // numero da linha
    int tabela[LIN][COL]; // tabela de tabuada

    // preenche a tabela

    for (y=0; y < LIN; y+=1)
        for(x=0; x < COL; x+=1)
            tabela[y][x] = y*x;

    cout << "\n          Tabela de Multiplicacao\n";
```

```
// Imprime o numero das colunas

cout << setw(6) << 0;
for (x=1; x < COL; x+=1)
    cout << setw(3) << x;
cout << endl;

// Imprime uma linha horizontal
cout << "  ";
```

```
for (x=0; x < 3*COL; x+=1)
    cout << "-";
cout << endl;

// Imprime as linhas da tablea.
// Cada linha a precedida pelo indice de linha e uma barra vertical

for (y=0; y < LIN; y+=1) {
    cout << setw(2) << y << "|";
    for(x=0; x < COL; x+=1)
        cout << setw(3) << tabela[y][x];
    cout << endl;
}
}
```



Atividades

Exercícios de Vetores:

1. Escreva uma função em C++ que receba um vetor de inteiros e retorne a soma de todos os elementos do vetor.
2. Crie uma função para encontrar o maior elemento em um vetor de doubles.
3. Implemente uma função que receba dois vetores de inteiros do mesmo tamanho e retorne um terceiro vetor contendo a soma dos elementos correspondentes dos dois vetores.
4. Escreva um programa que verifique se um vetor de inteiros está ordenado de forma crescente.
5. Crie uma função que receba um vetor de inteiros e um número inteiro e retorne verdadeiro se o número estiver presente no vetor, e falso caso contrário.
6. Implemente uma função que receba um vetor de inteiros e retorne um novo vetor contendo apenas os números pares do vetor original.
7. Escreva uma função que receba um vetor de strings e retorne a string mais longa presente no vetor.
8. Crie um programa que multiplique todos os elementos de um vetor de doubles por um valor especificado.

Atividades

Exercícios de Matrizes:

1. Implemente uma função em C++ para somar duas matrizes de inteiros.
2. Escreva um programa que verifique se uma matriz quadrada é simétrica em relação à sua diagonal principal.
3. Crie uma função para encontrar o menor elemento em uma matriz de floats.
4. Implemente uma função que receba uma matriz de inteiros e retorne verdadeiro se ela for uma matriz identidade, e falso caso contrário.
5. Escreva um programa que calcule o produto de duas matrizes.
6. Crie uma função que receba uma matriz de inteiros e retorne a soma dos elementos de cada linha como um vetor de inteiros.
7. Implemente um programa que determine se uma matriz é esparsa (possui a maioria dos elementos iguais a zero).
8. Escreva uma função para calcular a transposta de uma matriz quadrada de inteiros.